Maîtriser les expressions régulières

Lino Galiana
Version du 16 octobre 2026

Les expressions régulières fournissent un cadre très pratique pour manipuler de manière flexible des données textuelles. Elles sont très utiles notamment pour les tâches de traitement naturel du langage (**NLP**) ou le nettoyage de données textuelles.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Introduction **2**
2 Principe **2**
2.1 Classes de caractères 3
2.2 Quantifieurs 4
2.3 Aide-mémoire 5
3 Principales fonctions de `re` **8**
4 Généralisation avec `Pandas` **10**
5 Pour en savoir plus **11**
6 Exercices supplémentaires **11**
6.1 Extraction d'adresses email 11
6.2 Extraire des années depuis un `DataFrame Pandas` 12
Informations additionnelles 14

Compétences à l'issue de ce chapitre

- Comprendre l'utilité des expressions régulières (regex) pour manipuler des données textuelles de manière flexible, notamment au-delà des limitations de méthodes simples comme `str.find` ;
- Maîtriser les concepts fondamentaux des regex : classes de caractères (ex. `[a-z]`), quantifieurs (`?`, `*`, `+`, `{}`), ainsi que l'utilisation d'ancres et de méta-caractères ;
- Utiliser les principales fonctions du module `re` de Python (`findall`, `search`, `match`, `sub`, `finditer`, etc.) pour rechercher, extraire ou remplacer des motifs dans des chaînes ;
- Apprendre à appliquer efficacement les regex à des données tabulaires en utilisant l'API vectorisée de `Pandas` (`str.contains`, `str.extract`, `str.findall`, `str.replace`, `str.count`) ;
- Mettre en pratique les compétences acquises à travers des exercices ciblés : extraction de dates dans une chaîne et extraction d'adresses email ou d'années de publication depuis un DataFrame.

1 Introduction

`Python` offre énormément de fonctionalités très pratiques pour la manipulation de données textuelles. C'est l'une des raisons de son succès dans la communauté du traitement automatisé du langage (NLP, voir partie dédiée).

Dans les chapitres précédents, nous avons parfois été amenés à chercher des éléments textuels basiques. Cela était possible avec la méthode `str.find` du package `Pandas` qui constitue une version vectorisée de la méthode `find` de base. Nous avons d'ailleurs pu utiliser cette dernière directement, notamment lorsqu'on a fait du *web scraping*.

Cependant, cette fonction de recherche trouve rapidement ses limites. Par exemple, si on désire trouver à la fois les occurrences d'un terme au singulier et au pluriel, il sera nécessaire d'utiliser au moins deux fois la méthode `find`. Pour des verbes conjugués, cela devient encore plus complexe, en particulier si ceux-ci changent de forme selon le sujet.

Pour des expressions compliquées, il est conseillé d'utiliser les **expressions régulières**, ou *« regex »*. C'est une fonctionnalité qu'on retrouve dans beaucoup de langages. C'est une forme de grammaire qui permet de rechercher des expressions.

Une partie du contenu de cette partie est une adaptation de la documentation collaborative sur `R` nommée `utilitR` à laquelle j'ai participé. Ce chapitre reprend aussi du contenu du livre *R for Data Science* qui présente un chapitre très pédagogique sur les regex.

Nous allons utiliser le *package* `re` pour illustrer nos exemples d'expressions régulières. Il s'agit du package de référence, qui est utilisé, en arrière-plan, par `Pandas` pour vectoriser les recherches textuelles.

```
import re
import pandas as pd
```

Tip

Les expressions régulières (*regex*) sont notoirement difficiles à maîtriser. Il existe des outils qui facilitent le travail avec les expressions régulières.

- L'outil de référence pour ceci est [https://regex101.com/] qui permet de tester des `regex` en `Python` tout en ayant une explication qui accompagne ce test
- De même pour ce site qui comporte une cheat sheet en bas de la page.
- Les jeux de Regex Crossword permettent d'apprendre les expressions régulières en s'amusant

Il peut être pratique de demander à des IA assistantes, comme `Github Copilot` ou `ChatGPT`, une première version d'une regex en expliquant le contenu qu'on veut extraire. Cela peut faire économiser pas mal de temps, sauf quand l'IA fait preuve d'une confiance excessive et vous propose avec aplomb une regex totalement fausse…

2 Principe

Les expressions régulières sont un outil permettant de décrire un ensemble de chaînes de caractères possibles selon une syntaxe précise, et donc de définir un motif (ou `pattern`). Les expressions régulières servent par exemple lorsqu'on veut extraire une partie d'une chaîne de caractères, ou remplacer une partie d'une chaîne de caractères. Une expression régulière prend la

forme d'une chaîne de caractères, qui peut contenir à la fois des éléments littéraux et des caractères spéciaux qui ont un sens logique.

Par exemple, `"ch.+n"` est une expression régulière qui décrit le motif suivant : la chaîne littérale `ch`, suivi de n'importe quelle chaîne d'au moins un caractère (`.+`), suivie de la lettre `n`. Dans la chaîne `"J'ai un chien."`, la sous-chaîne `"chien"` correspond à ce motif. De même pour `"chapeau ron"` dans `"J'ai un chapeau rond"`. En revanche, dans la chaîne `"La soupe est chaude."`, aucune sous-chaîne ne correpsond à ce motif (car aucun `n` n'apparaît après le `ch`).

Pour s'en convaincre, nous pouvons déjà regarder les deux premiers cas:

```
pattern = "ch.+n"
print(re.search(pattern, "La soupe est chaude."))
```

```
None
```

La regex précédente comportait deux types de caractères:

- les *caractères littéraux* : lettres et nombres qui sont reconnus de manière littérale
- les *méta-caractères* : symboles qui ont un sens particulier dans les regex.

Les principaux *méta-caractères* sont `.`, `+`, `*`, `[`, `]`, `^` et `$` mais il en existe beaucoup d'autres. Parmi cet ensemble, on utilise principalement les quantifieurs (`.`, `+`, `*`...), les classes de caractères (ensemble qui sont délimités par `[` et `]`) ou les ancres (`^`, `$`...)

Dans l'exemple précédent, nous retrouvions deux quantifieurs accolés `.+`. Le premier (`.`) signifie n'importe quel caractère[1]. Le deuxième (`+`) signifie *« répète le pattern précédent »*. Dans notre cas, la combinaison `.+` permet ainsi de répéter n'importe quel caractère avant de trouver un *n*. Le nombre de fois est indeterminé : cela peut ne pas être pas nécessaire d'intercaler des caractères avant le *n* ou cela peut être nécessaire d'en intercepter plusieurs :

```
print(re.search(pattern, "J'ai un chino"))
print(re.search(pattern, "J'ai un chiot très mignon."))
```

```
<re.Match object; span=(8, 12), match='chin'>
<re.Match object; span=(8, 25), match='chiot très mignon'>
```

2.1 Classes de caractères

Lors d'une recherche, on s'intéresse aux caractères et souvent aux classes de caractères : on cherche un chiffre, une lettre, un caractère dans un ensemble précis ou un caractère qui n'appartient pas à un ensemble précis. Certains ensembles sont prédéfinis, d'autres doivent être définis à l'aide de crochets.

Pour définir un ensemble de caractères, il faut écrire cet ensemble entre crochets. Par exemple, `[0123456789]` désigne un chiffre. Comme c'est une séquence de caractères consécutifs, on peut résumer cette écriture en `[0-9]`.

Par exemple, si on désire trouver tous les *pattern* qui commencent par un `c` suivi d'un `h` puis d'une voyelle (a, e, i, o, u), on peut essayer cette expression régulière.

```
re.findall("[c][h][aeiou]", "chat, chien, veau, vache, chèvre")
```

```
['cha', 'chi', 'che']
```

[1] N'importe quel caractère à part le retour à la ligne (`\n`). Ceci est à garder en tête, j'ai déjà perdu des heures à chercher pourquoi mon `.` ne capturait pas ce que je voulais qui s'étalait sur plusieurs lignes...

Il serait plus pratique d'utiliser `Pandas` dans ce cas pour isoler les lignes qui répondent à la condition logique (en ajoutant les accents qui ne sont pas compris sinon):

```
import pandas as pd
txt = pd.Series("chat, chien, veau, vache, chèvre".split(", "))
txt.str.match("ch[aeéèiou]")
```

```
0     True
1     True
2    False
3    False
4     True
dtype: bool
```

Cependant, l'usage ci-dessus des classes de caractères n'est pas le plus fréquent. On privilégie celles-ci pour identifier des pattern complexe plutôt qu'une suite de caractères littéraux. Les tableaux d'aide mémoire illustrent une partie des classes de caractères les plus fréquentes (`[:digit:]` ou `\d`...)

2.2 Quantifieurs

Nous avons rencontré les quantifieurs avec notre première expression régulière. Ceux-ci contrôlent le nombre de fois qu'un *pattern* est rencontré.

Les plus fréquents sont:

- `?` : 0 ou 1 match ;
- `+` : 1 ou plus de matches ;
- `*` : 0 or more matches.

Par exemple, `colou?r` permettra de matcher à la fois l'écriture américaine et anglaise

```
re.findall("colou?r", "Did you write color or colour?")
```

```
['color', 'colour']
```

Ces quantifiers peuvent bien sûr être associés à d'autres types de caractères, notamment les classes de caractères. Cela peut être extrêmement pratique. Par exemple, `\d+` permettra de capturer un ou plusieurs chiffres, `\s?` permettra d'ajouter en option un espace, `[\w]{6,8}` un mot entre six et huit lettres qu'on écrira...

Il est aussi possible de définir le nombre de répétitions avec `{}`:

- `{n}` matche exactement n fois ;
- `{n,}` matche au moins n fois ;
- `{n,m}` matche entre n et m fois.

Cependant, la répétition des termes ne s'applique par défaut qu'au dernier caractère précédent le quantifier. On peut s'en convaincre avec l'exemple ci-dessus:

```
print(re.match("toc{4}","toctoctoctoc"))
```

```
None
```

Pour pallier ce problème, il existe les parenthèses. Le principe est le même qu'avec les règles numériques: les parenthèses permettent d'introduire une hiérarchie. Pour reprendre l'exemple précédent, on obtient bien le résultat attendu grâce aux parenthèses:

```
print(re.match("(toc){4}","toctoctoctoc"))
print(re.match("(toc){5}","toctoctoctoc"))
print(re.match("(toc){2,4}","toctoctoctoc"))
```

```
<re.Match object; span=(0, 12), match='toctoctoctoc'>
None
<re.Match object; span=(0, 12), match='toctoctoctoc'>
```

Note

L'algorithme des expressions régulières essaye toujours de faire correspondre le plus grand morceau à l'expression régulière.

Par exemple, soit une chaine de caractère HTML:

```
s = "<h1>Super titre HTML</h1>"
```

L'expression régulière `re.findall("<.*>", s)` correspond, potentiellement, à trois morceaux :

- `<h1>`
- `</h1>`
- `<h1>Super titre HTML</h1>`

C'est ce dernier qui sera choisi, car le plus grand. Pour sélectionner le plus petit, il faudra écrire les multiplicateurs comme ceci : `*?`, `+?`. En voici quelques exemples:

```
s = "<h1>Super titre HTML</h1>\n<p><code>Python</code> est un langage très flexible</p>"
print(re.findall("<.*>", s))
print(re.findall("<p>.*</p>", s))
print(re.findall("<p>.*?</p>", s))
print(re.compile("<.*?>").findall(s))
```

```
['<h1>Super titre HTML</h1>', '<p><code>Python</code> est un langage très flexible</p>']
['<p><code>Python</code> est un langage très flexible</p>']
['<p><code>Python</code> est un langage très flexible</p>']
['<h1>', '</h1>', '<p>', '<code>', '</code>', '</p>']
```

2.3 Aide-mémoire

Le tableau ci-dessous peut servir d'aide-mémoire sur les regex:

Expression régulière	Signification
`"^"`	Début de la chaîne de caractères
`"$"`	Fin de la chaîne de caractères
`"\\."`	Un point
`"."`	N'importe quel caractère
`".+"`	N'importe quelle suite de caractères non vide
`".*"`	N'importe quelle suite de caractères, éventuellement vi
`"[:alnum:]"`	Un caractère alphanumérique
`"[:alpha:]"`	Une lettre
`"[:digit:]"`	Un chiffre

Expression régulière	Signification
`"[:lower:]"`	Une lettre minuscule
`"[:punct:]"`	Un signe de ponctuation
`"[:space:]"`	un espace
`"[:upper:]"`	Une lettre majuscule
`"[[:alnum:]]+"`	Une suite d'au moins un caractère alphanumérique
`"[[:alpha:]]+"`	Une suite d'au moins une lettre
`"[[:digit:]]+"`	Une suite d'au moins un chiffre
`"[[:lower:]]+"`	Une suite d'au moins une lettre minuscule
`"[[:punct:]]+"`	Une suite d'au moins un signe de ponctuation
`"[[:space:]]+"`	Une suite d'au moins un espace
`"[[:upper:]]+"`	Une suite d'au moins une lettre majuscule
`"[[:alnum:]]*"`	Une suite de caractères alphanumériques, éventuellement vide
`"[[:alpha:]]*"`	Une suite de lettres, éventuellement vide
`"[[:digit:]]*"`	Une suite de chiffres, éventuellement vide
`"[[:lower:]]*"`	Une suite de lettres minuscules, éventuellement vide
`"[[:upper:]]*"`	Une suite de lettres majuscules, éventuellement vide
`"[[:punct:]]*"`	Une suite de signes de ponctuation, éventuellement vide
`"[^[:alpha:]]+"`	Une suite d'au moins un caractère autre qu'une lettre
`"[^[:digit:]]+"`	Une suite d'au moins un caractère autre qu'un chiffre
`"|"`	L'une des expressions `x` ou `y` est présente
`[abyz]`	Un seul des caractères spécifiés
`[abyz]+`	Un ou plusieurs des caractères spécifiés (éventuellement répétés)
`[^abyz]`	Aucun des caractères spécifiés n'est présent

Certaines classes de caractères bénéficient d'une syntaxe plus légère car elles sont très fréquentes. Parmi-celles:

Expression régulière	Signification
`\d`	N'importe quel chiffre
`\D`	N'importe quel caractère qui n'est pas un caractère
`\s`	N'importe quel espace (espace, tabulation, retour à la ligne)
`\S`	N'importe quel caractère qui n'est pas un espace
`\w`	N'importe quel type de mot (lettres et nombres)
`\W`	N'importe quel ensemble qui n'est pas un mot (lettres et nombres)

Dans l'exercice suivant, vous allez pouvoir mettre en pratique les exemples précédents sur une `regex` un peu plus complète. Cet exercice ne nécessite pas la connaissance des subtilités du *package* `re`, vous n'aurez besoin que de `re.findall`.

Cet exercice utilisera la chaine de caractère suivante :

```
s = """date 0 : 14/9/2000
date 1 : 20/04/1971     date 2 : 14/09/1913     date 3 : 2/3/1978
date 4 : 1/7/1986     date 5 : 7/3/47     date 6 : 15/10/1914
date 7 : 08/03/1941     date 8 : 8/1/1980     date 9 : 30/6/1976"""
s
```

```
'date 0 : 14/9/2000\ndate 1 : 20/04/1971     date 2 : 14/09/1913     date 3 : 2/3/1978\ndate 4 :
1/7/1986     date 5 : 7/3/47     date 6 : 15/10/1914\ndate 7 : 08/03/1941     date 8 : 8/1/1980
date 9 : 30/6/1976'
```

Exercice 1

1. On va d'abord s'occuper d'extraire le jour de naissance.
   - Le premier chiffre du jour est 0, 1, 2 ou 3. Traduire cela sous la forme d'une séquence `[X-X]`
   - Le deuxième chiffre du jour est lui entre 0 et 9. Traduire cela sous la séquence adéquate
   - Remarquez que le premier jour est facultatif. Intercaler entre les deux classes de caractère adéquate le quantifieur qui convient
   - Ajouter le slash à la suite du motif
   - Tester avec `re.findall`. Vous devriez obtenir beaucoup plus d'échos que nécessaire. C'est normal, à ce stade la regex n'est pas encore finalisée
2. Suivre la même logique pour les mois en notant que les mois du calendrier grégorien ne dépassent jamais la première dizaine. Tester avec `re.findall`
3. De même pour les années de naissance en notant que jusqu'à preuve du contraire, pour des personnes vivantes aujourd'hui, les millénaires concernés sont restreints. Tester avec `re.findall`
4. Cette regex n'est pas naturelle, on pourrait très bien se satisfaire de classes de caractères génériques `\d` même si elles pourraient, en pratique, nous sélectionner des dates de naissance non possibles (`43/78/4528` par exemple). Cela permettrait d'alléger la regex afin de la rendre plus intelligible. Ne pas oublier l'utilité des quantifieurs.
5. Comment adapter la regex pour qu'elle soit toujours valide pour nos cas mais permette aussi de capturer les dates de type `YYYY/MM/DD` ? Tester sur `1998/07/12`

A l'issue de la question 1, vous devriez avoir ce résultat :

```
['14/',
 '9/',
 '20/',
 '04/',
 '14/',
 '09/',
 '2/',
 '3/',
 '1/',
 '7/',
 '7/',
 '3/',
 '15/',
 '10/',
 '08/',
 '03/',
 '8/',
 '1/',
 '30/',
 '6/']
```

A l'issue de la question 2, vous devriez avoir ce résultat, qui commence à prendre forme:

```
['14/9',
 '20/04',
 '14/09',
 '2/3',
 '1/7',
 '7/3',
 '15/10',
 '08/03',
 '8/1',
 '30/6']
```

A l'issue de la question 3, on parvient bien à extraire les dates :

```
['14/9/2000',
 '20/04/1971',
 '14/09/1913',
 '2/3/1978',
 '1/7/1986',
 '7/3/47',
 '15/10/1914',
 '08/03/1941',
 '8/1/1980',
 '30/6/1976']
```

Si tout va bien, à la question 5, votre regex devrait fonctionner:

```
['14/9/2000',
 '20/04/1971',
 '14/09/1913',
 '2/3/1978',
 '1/7/1986',
 '7/3/47',
 '15/10/1914',
 '08/03/1941',
 '8/1/1980',
 '30/6/1976',
 '1998/07/12']
```

3 Principales fonctions de `re`

Voici un tableau récapitulatif des principales fonctions du package `re` suivi d'exemples.

Nous avons principalement utilisé jusqu'à présent `re.findall` qui est l'une des fonctions les plus pratiques du *package*. `re.sub` et `re.search` sont également bien pratiques. Les autres sont moins vitales mais peuvent dans des cas précis être utiles.

Fonction	Objectif
`re.match(<regex>, s)`	Trouver et renvoyer le **premier** *match* de l'expression régulière `<regex>` **à partir du début** du *string* `s`
`re.search(<regex>, s)`	Trouver et renvoyer le **premier** *match* de l'expression régulière `<regex>` **quelle que soit sa position** dans le *string* `s`

Fonction	Objectif
`re.finditer(<regex>, s)`	Trouver et renvoyer un itérateur stockant tous les *matches* de l'expression régulière `<regex>` **quelle que soit leur(s) position(s)** dans le *string* `s`. En général, on effectue ensuite une boucle sur cet itérateur
`re.findall(<regex>, s)`	Trouver et renvoyer **tous les *matches*** de l'expression régulière `<regex>` **quelle que soit leur(s) position(s)** dans le *string* `s` sous forme de **liste**
`re.sub(<regex>, new_text, s)`	Trouver et **remplacer tous** les *matches* de l'expression régulière `<regex>` **quelle que soit leur(s) position(s)** dans le *string* `s`

Pour illustrer ces fonctions, voici quelques exemples:

`re.match` ne peut servir qu'à capturer un *pattern* en début de *string*. Son utilité est donc limitée. Capturons néanmoins `toto` :

```
re.match("(to){2}", "toto à la plage")
```

```
<re.Match object; span=(0, 4), match='toto'>
```

`re.search` est plus puissant que `re.match`, on peut capturer des termes quelle que soit leur position dans un *string*. Par exemple, pour capturer *age* :

```
re.search("age", "toto a l'age d'aller à la plage")
```

```
<re.Match object; span=(9, 12), match='age'>
```

Et pour capturer exclusivement *« age »* en fin de *string* :

```
re.search("age$", "toto a l'age d'aller à la plage")
```

```
<re.Match object; span=(28, 31), match='age'>
```

`re.finditer` est, à mon avis, moins pratique que `re.findall`. Son utilité principale par rapport à `re.findall` est de capturer la position dans un champ textuel:

```
s = "toto a l'age d'aller à la plage"
for match in re.finditer("age", s):
    start = match.start()
    end = match.end()
    print(f'String match "{s[start:end]}" at {start}:{end}')
```

```
String match "age" at 9:12
String match "age" at 28:31
```

`re.sub` permet de capturer et remplacer des expressions. Par exemple, remplaçons *« age »* par *« âge »*. Mais attention, il ne faut pas le faire lorsque le motif est présent dans *« plage »*. On va donc mettre une condition négative: capturer *« age »* seulement s'il n'est pas en fin de *string* (ce qui se traduit en *regex* par `?!$`)

```
re.sub("age(?!$)", "âge", "toto a l'age d'aller à la plage")
```

```
"toto a l'âge d'aller à la plage"
```

Quand utiliser re.compile et les raw strings ?

`re.compile` peut être intéressant lorsque vous utilisez une expression régulière plusieurs fois dans votre code. Cela permet de compiler l'expression régulière en un objet reconnu par `re`, ce qui peut être plus efficace en termes de performance lorsque l'expression régulière est utilisée à plusieurs reprises ou sur des données volumineuses.

Les chaînes brutes (*raw string*) sont des chaînes de caractères spéciales en `Python`, qui commencent par `r`. Par exemple `r"toto à la plage"`. Elles peuvent être intéressantes pour éviter que les caractères d'échappement ne soient interprétés par `Python` Par exemple, si vous voulez chercher une chaîne qui contient une barre oblique inverse `\` dans une chaîne, vous devez utiliser une chaîne brute pour éviter que la barre oblique inverse ne soit interprétée comme un caractère d'échappement (`\t`, `\n`, etc.). Le testeur https://regex101.com/ suppose d'ailleurs que vous utilisez des *raw string*, cela peut donc être utile de s'habituer à les utiliser.

4 Généralisation avec `Pandas`

Les méthodes de `Pandas` sont des extensions de celles de `re` qui évitent de faire une boucle pour regarder, ligne à ligne, une regex. En pratique, lorsqu'on traite des `DataFrames`, on utilise plutôt l'API Pandas que `re`. Les codes de la forme `df.apply(lambda x: re.<fonction>(<regex>,x), axis = 1)` sont à bannir car très peu efficaces.

Les noms changent parfois légèrement par rapport à leur équivalent `re`.

Méthode	Description
`str.count()`	Compter le nombre d'occurrences du *pattern* dans chaque ligne
`str.replace()`	Remplacer le *pattern* par une autre valeur. Version vectorisée de `re.sub()`
`str.contains()`	Tester si le *pattern* apparaît, ligne à ligne. Version vectorisée de `re.search()`
`str.extract()`	Extraire les groupes qui répondent à un *pattern* et les renvoyer dans une colonne
`str.findall()`	Trouver et renvoyer toutes les occurrences d'un *pattern*. Si une ligne comporte plusieurs échos, une liste est renvoyée. Version vectorisée de `re.findall()`

A ces fonctions, s'ajoutent les méthodes `str.split()` et `str.rsplit()` qui sont bien pratiques.

On peut compter le nombre de fois qu'un *pattern* apparaît avec `str.count`

```
df = pd.DataFrame({"a": ["toto", "titi"]})
df['a'].str.count("to")
```

```
0    2
1    0
Name: a, dtype: int64
```

Remplaçons le motif *« ti »* en fin de phrase

```
df = pd.DataFrame({"a": ["toto", "titi"]})
df['a'].str.replace("ti$", " punch")
```

```
0    toto
1    titi
Name: a, dtype: str
```

Vérifions les cas où notre ligne termine par *« ti »* :

```
df = pd.DataFrame({"a": ["toto", "titi"]})
df['a'].str.contains("ti$")
```

```
0    False
1     True
Name: a, dtype: bool
```

```
df = pd.DataFrame({"a": ["toto", "titi"]})
df['a'].str.findall("to")
```

```
0    [to, to]
1          []
Name: a, dtype: object
```

5 Pour en savoir plus

- documentation collaborative sur `R` nommée `utilitR`
- *R for Data Science*
- *Regular Expression HOWTO* dans la documentation officielle de `Python`
- L'outil de référence [https://regex101.com/] pour tester des expressions régulières
- Ce site qui comporte une cheat sheet en bas de la page.
- Les jeux de Regex Crossword permettent d'apprendre les expressions régulières en s'amusant

6 Exercices supplémentaires

6.1 Extraction d'adresses email

Il s'agit d'un usage classique des *regex*

```
text_emails = 'Hello from toto@gmail.com to titi.grominet@yahoo.com about the meeting @2PM'
```

Exercice 2: extraction d'adresses email

Utiliser la structure d'une adresse mail `[XXXX]@[XXXX]` pour récupérer ce contenu

```
['toto@gmail.com', 'titi.grominet@yahoo.com']
```

6.2 Extraire des années depuis un `DataFrame Pandas`

L'objectif général de l'exercice est de nettoyer des colonnes d'un DataFrame en utilisant des expressions régulières.

Exercice 3

La base en question contient des livres de la British Library et quelques informations les concernant. Le jeu de données est disponible ici : https://raw.githubusercontent.com/realpython/python-data-cleaning/master/Datasets/BL-Flickr-Images-Book.csv

La colonne « Date de Publication » n'est pas toujours une année, il y a parfois d'autres informations. Le but de l'exercice est d'avoir **une date de publication du livre propre** et de regarder la **distribution des années de publications**.

Pour ce faire, vous pouvez :

- Soit choisir de réaliser l'exercice sans aide. Votre **lecture de l'énoncé s'arrête donc ici**. Vous devez alors faire attention à bien regarder vous-même la base de données et la transformer avec attention.
- Soit suivre les différentes étapes qui suivent pas à pas.

Version guidée 👇

1. Lire les données depuis l'url `https://raw.githubusercontent.com/realpython/python-data-cleaning/master/Datasets/BL-Flickr-Images-Book.csv`. Attention au séparateur
2. Ne garder que les colonnes `['Identifier', 'Place of Publication', 'Date of Publication', 'Publisher', 'Title', 'Author']`
3. Observer la colonne *"Date of Publication"* et remarquer le problème sur certaines lignes (par exemple la ligne 13)
4. Commencez par regarder le nombre d'informations manquantes. On ne pourra pas avoir mieux après la regex, et normalement on ne devrait pas avoir moins...
5. Déterminer la forme de la regex pour une date de publication. A priori, il y a 4 chiffres qui forment une année. Utiliser la méthode `str.extract()` avec l'argument `expand = False` (pour ne conserver que la première date concordant avec notre *pattern*)?
6. On a 2 `NaN` qui n'étaient pas présents au début de l'exercice. Quels sont-ils et pourquoi ?
7. Quelle est la répartition des dates de publications dans le jeu de données ? Vous pouvez par exemple afficher un histogramme grâce à la méthode `plot` avec l'argument `kind ="hist"`.

Voici par exemple le problème qu'on demande de détecter à la question 3 :

	Date of Publication	Title
13	1839, 38-54	De Aardbol. Magazijn van hedendaagsche land- e...
14	1897	Cronache Savonesi dal 1500 al 1570 ... Accresc...
15	1865	See-Saw; a novel ... Edited [or rather, writte...

	Date of Publication	Title
16	1860-63	Géodésie d'une partie de la Haute Éthiopie,...
17	1873	[With eleven maps.]
18	1866	[Historia geográfica, civil y politica de la ...
19	1899	The Crisis of the Revolution, being the story ...

A la question 4, on obtient la réponse

```
np.int64(181)
```

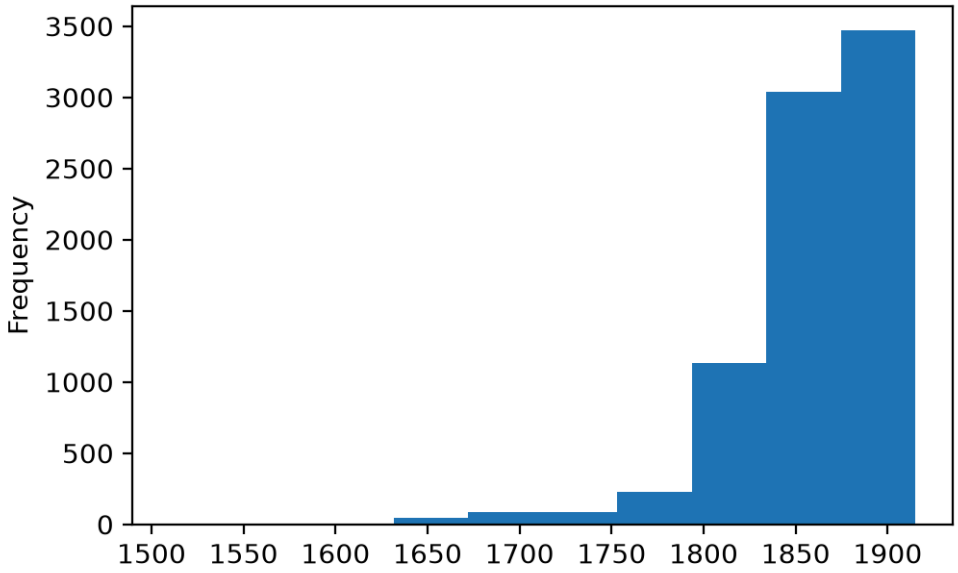
Grâce à notre regex (question 5), on obtient ainsi un `DataFrame` plus conforme à nos attentes

	Date of Publication	year
0	1879 [1878]	1879
7	NaN	NaN
13	1839, 38-54	1839
16	1860-63	1860
23	1847, 48 [1846-48]	1847
...	...	...
8278	1883, [1884]	1883
8279	1898-1912	1898
8283	1831, 32	1831
8284	[1806]–22	1806
8286	1834-43	1834

Quant aux nouveaux `NaN`, il s'agit de lignes qui ne contenaient pas de chaînes de caractères qui ressemblaient à des années :

	Date of Publication	year
1081	112. G. & W. B. Whittaker	NaN
7391	17 vols. University Press	NaN

Enfin, on obtient l'histogramme suivant des dates de publications:



Informations additionnelles

i Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru==0.7.3",
    "markdown≥3.8",
    "nbclient≥0.10.0",
    "nbformat≥5.10.4",
    "nltk≥3.9.1",
    "openpyxl≥3.1.5",
    "pandas≥3.0",
    "pip≥25.1.1",
    "plotly≥6.1.2",
    "plotnine≥0.15",
    "polars≥1.8.2",
    "py7zr≥1.1.3",
    "pyarrow≥17.0.0",
    "pynsee≥0.1.8",
    "python-dotenv≥1.0.1",
    "python-frontmatter≥1.1.0",
    "pywaffle≥1.1.1",
    "requests≥2.32.3",
    "scikit-image≥0.24.0",
    "scikit-learn≥1.8.0",
    "scipy≥1.13.0",
    "seaborn≥0.13.2",
    "selenium<4.39.0",
    "spacy≥3.8.4",
    "webdriver-manager≥4.0.2",
```

```
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.